

Lab 04 Template

Part 01

1. Screenshot of encrypted message from `part01_plaintext.txt` by using a 4-bit LFSR with the seed "1001" with the first and last bit XORd with each other. Use `xxd -b part01_ciphertext.bin` to display both the binary and ASCII. (10 points)

```
cpre3310@cpre3310:~/homework/lab04$ cd part01
cpre3310@cpre3310:~/homework/lab04/part01$ python3 part01_02_skel.py part01_plaintext.txt ascii part01_ciphertext.bin binary part01_keyStream.bin binary
Do you want to 1) encrypt/decrypt 2) check encryption or 3) conduct cryptanalysis?1
Enter starting values: 1001
Do you need to to little endian ? y/n: y

inputDataInBits:
[0 0 1 0 0 0 1 0 1 0 0 1 0 1 1 1 0 0 0 1 0 0 1 1 1 0 0 0 0 0 0 1 0 0 1 0 0 1 1
1 1 0 1 1 1 1 1 0 1 1 1 0 1 1 0 1 1 1 1 0 0 0 0 0 0 1 0 0 1 0 1 0 0 1 1 0 0 1
1 1 0 1 1 0 1 1 1 0 0 0 1 1 0 0 1 0 0 1 1 1 1 0 1 0 0 1 1 1 1 0 0 0 0 0 0 1 1 1
0 0 0 1 0 1 1 1 0 0 0 0 0 0 1 0 0 0 0 1 0 1 1 1 0 0 0 0 1 0 1 1 0 1 0 0 1
0 1 1 0 1 1 0 0 1 1 1 1 0 0 0 0 0 1 0 0 1 1 0 0 0 1 1 0 1 1 1 1 0 1 1 0 0
1 0 0 1 1 1 0 0 1 0 0 1 1 1 0 1 0 1 0 0 1 1 0 1 1 0 0 0 1 1 0 0 0 1 0 1 1
1 0 0 0 1 1 0 1 1 1 0 1 0 0 1 1 1 1 0 1 1 1 1 1 0 0 0 0 0 0 0 1 0 0 0 0 1
0 1 0 1 0 0 0 0 1 0 1 1 0 1 0 1 0 0 1 1 0 0 1 0 0 1 1 1 0 1 0 1 0 0 1 1 0
0 0 0 0 0 1 0 0 1 0 0 1 0 1 1 0 1 1 0 0 1 1 1 0 0 0 0 0 0 1 0 0 1 1 1 1 0
1 1 0 0 1 1 1 0 1 1 0 0 0 1 1 0 1 1 0 1 0 0 1 1 1 1 0 0 0 0 0 0 1 0 0 1 1
1 1 0 1 1 0 0 1 1 1 0 1 1 0 1 0 1 0 0 1 1 0 0 0 0 0 0 1 0 0 1 1 1 0 1 1 1
0 1 0 0 0 1 1 0 1 0 0 1 1 1 1 0 0 0 0 0 0 1 0 0 0 0 1 0 1 1 1 0 1 1 1 1
0 1 1 0 0 0 0 0 1 0 0 0 1 1 0 0 1 1 0 1 0 0 1 0 1 1 0 0 0 1 1 1 0 1 1 0 0
0 1 0 0 1 1 0 0 0 0 0 1 0 0 1 1 1 1 0 1 1 0 1 0 1 0 1 1 1 0 0 0 1 0 1 1
1 0 0 1 1 1 0 1 0 0 0 1 1 1 0 1 0 0 0 1 1 1 0 1 0 0 0 1 0 1 0 0 0 0]

keyStreamBits:
[1 0 0 1 0 0 0 1 1 1 1 0 1 0 1 1 0 0 1 0 0 0 1 1 1 1 0 1 0 1 1 0 0 1 0 0 0
1 1 1 1 0 1 0 1 1 0 0 1 0 0 0 1 1 1 1 0 1 0 1 1 0 0 1 0 0 0 1 1 1 1 0 1 0
1 1 0 0 1 0 0 0 1 1 1 1 0 1 0 1 1 0 0 1 0 0 0 1 1 1 1 0 1 0 1 1 0 0 1 0 0
0 1 1 1 1 0 1 0 1 1 0 0 1 0 0 0 1 1 1 1 0 1 0 1 1 0 0 1 0 0 0 1 1 1 1 0 1
0 1 1 0 0 1 0 0 0 1 1 1 1 0 1 0 1 1 0 0 1 0 0 0 1 1 1 1 0 1 0 1 1 0 0 1 0
0 0 1 1 1 1 0 1 0 1 1 0 0 1 0 0 0 1 1 1 1 0 1 0 1 1 0 0 1 0 0 0 1 1 1 1 0
1 0 1 1 0 0 1 0 0 0 1 1 1 1 0 1 0 1 1 0 0 1 0 0 0 1 1 1 1 0 1 0 1 1 0 0 1
0 0 0 1 1 1 1 0 1 0 1 1 0 0 1 0 0 0 1 1 1 1 0 1 0 1 1 0 0 1 0 0 0 1 1 1 1
0 1 0 1 1 0 0 1 0 0 0 1 1 1 1 0 1 0 1 1 0 0 1 0 0 0 1 1 1 1 0 1 0 1 1 0 0
1 0 0 0 1 1 1 1 0 1 0 1 1 0 0 1 0 0 0 1 1 1 1 0 1 0 1 1 0 0 1 0 0 0 1 1 1
1 0 1 0 1 1 0 0 1 0 0 0 1 1 1 1 0 1 0 1 1 0 0 1 0 0 0 1 1 1 1 0 1 0 1 1 0
0 1 0 0 0 1 1 1 1 0 1 0 1 1 0 0 1 0 0 0 1 1 1 1 0 1 0 1 1 0 0 1 0 0 0 1 1]
```

```

Feb 25 15:20
cpre3310@cpre3310: ~/homework/lab04/part01

1101100111011010100110000001001110111
0100001101001111000000100001011101111
0110000001000110011010010110011101100
0100110000001001111011010101110001011
1001110100011101000111010001010000]

keyStreamBits:
[1001000111101011001000111101011001000
1111010110010001111010110010001111010
1100100011110101100100011110101100100
0111101011001000111101011001000111101
0110010001111010110010001111010110010
0011110101100100011110101100100011110
1011001000111101011001000111101011001
0001111010110010001111010110010001111
0101100100011110101100100011110101100
1000111101011001000111101011001000111
10110010001111010110010001111010110
0100011110101100100011110101100100011
11011001000111101011001000111101011
0010001111010110010001111010110010001
1110101100100011110101100100011110]
resultXOR: [1011001101111101000001011101001011011
0010101101000100001010111011011100011
0001001111101100101010111001001100011
0110110111001010111000101001101010100
0000100010011010100001001001101011110
101000011111001001101110100010010101
001111110011010110110110111101111000
0100111000000111000011110001000101001
010111011001000011111000011100110010
0100000110011111110011010111001010100
0111010101010101111000010000110100001
0000010011100011100011010100111001100
101101100000001110001011110100000111
011011111101111101010101111000011010
0111011000111110110010110101001110]
cpre3310@cpre3310: ~/homework/lab04/part01

```

```

cpre3310@cpre3310:~/homework/lab04/part01$ xxd -b part01_ciphertext.bin
00000000: 11001101 10111110 10100000 01001011 10011011 01011010 ...K.Z
00000006: 10000100 10111010 00011101 00100011 11011111 01010100 ...#.T
0000000c: 00100111 01100011 11011011 10101001 10100011 10101100 'c....
00000012: 00000010 10010001 00010101 10010010 11110101 00001010 .....
00000018: 00111111 11011001 01000101 00101010 01111111 11010110 ?.E*..
0000001e: 10110110 11110111 10010000 00000011 10000111 01000111 .....G
00000024: 10010100 10111010 00010001 00111110 10011100 01001001 ...>.I
0000002a: 00110000 01111111 11010110 10101001 10111000 10101010 0.....
00000030: 00001110 11000010 01000010 10010000 11100011 01011000 ..B..X
00000036: 00111001 11010011 00000110 00111000 01111010 11000001 9..8z.
0000003c: 11101101 11110111 10101011 00011110 10010110 00011011 .....
00000042: 11011111 10110100 01110010 ..r
cpre3310@cpre3310:~/homework/lab04/part01$

```

Part 02

1. XOR the known plaintext with the first X bits of the ciphertext, where X is the number of bits in the known plaintext. Print out the 0s and 1s.
(5 points)

In order to recover the keystream prefix, I basically XORed the known plaintext with the X bits of the ciphertext, where X is basically the number of known plaintext. The formula that I used over here was $\text{Keystream} = \text{Known Plaintext} \oplus \text{Ciphertext}$ to get this output.

```

cpre3310@cpre3310:~/homework/lab04/part02$ python3 ../part01/part01_02_skel.py part02_knownPlainte
xt.txt ascii part02_ciphertext.bin binary part02_possible_keyStream.bin binary
Do you want to 1) encrypt/decrypt 2) check encryption or 3) conduct cryptanalysis?3
resultXOR: [0 0 1 1 0 1 0 1 1 1 1 0 0 0 1 0 0 1 1 0 1 0 1 1 1 1 0 0 0 1 0 0 1 1 0 1 0
1 1 1 1 0 0 0 1 0 0 1 1 0 1 0 1 1 1 1 0 0 0 1 0 0 1 1 0 1 0 1 1 1 1 0 0 0
1 0 0 1 1 0 1 0 1 1 1 1 1 0 0 0 1 0 0 1 1 0 1]
Potential Key Stream:
[0 0 1 1 0 1 0 1 1 1 1 0 0 0 1 0 0 1 1 0 1 0 1 1 1 1 0 0 0 1 0 0 1 1 0 1 0
1 1 1 1 0 0 0 1 0 0 1 1 0 1 0 1 1 1 1 0 0 0 1 0 0 1 1 0 1 0 1 1 1 1 0 0 0
1 0 0 1 1 0 1 0 1 1 1 1 1 0 0 0 1 0 0 1 1 0 1]
cpre3310@cpre3310:~/homework/lab04/part02$

```

2. Calculate the period by showing the number of bits that are repeated in the keystream.
(5 points)

Since the total number of bits that repeats after 15 digits (001101011110001), we can say that the period is 15 bits

3. Calculate the degree (Show your work step by step).
(5 points)

From the lecture we have learnt that $\text{period} = 2^n - 1$ and since we know that the bit repeats at every 15 bits, we can take out the answer in the following way:

$$\begin{aligned}
 15 &= 2^n - 1 \\
 16 &= 2^n \\
 n &= 4
 \end{aligned}$$

Hence, the degree is 4 over here.

4. Calculate the seed by pasting your keystream into a spreadsheet and tracing it back to the initial starting values.
(10 points)

A	B	C	D
1	1	0	0
0	1	1	0
1	0	1	1
0	1	0	1
1	0	1	0
1	1	0	1
1	1	1	0
1	1	1	1
0	1	1	1
0	0	1	1
0	0	0	1
1	0	0	0
	1	0	0
		1	0
			1

Over here, I first copied the 15 bits that I found earlier in a spreadsheet, where each row basically represents 1 register state of the 4-bit LFSR. Now, I basically shifted the bits forward in each column in order to determine which 2 registers were XORED to generate the next state and then I was able to determine the initial register values and since the first row displays the key bits 1100, we can say that our seed is 1100 now.

5. Use the spreadsheet from step 4 to find the LFSR configuration and show your work for each step.

(10 points)

Row 1 and 2:

Over here, in row 1 we can see that $1 \text{ XOR } 1$ and $0 \text{ XOR } 0$ both turn out to be 0 so our required registers could either be register[0] and register[1] or it could be register[2] and register[3]. We can give a similar reason for row 2.

Row 3:

Now, if we look at row 3, and the first bit of row 4, which is 0, we can say that our required registers will be register[2] and register[3], since if we XOR register[0] and register[1], we get 1 and not 0.

Row 4:

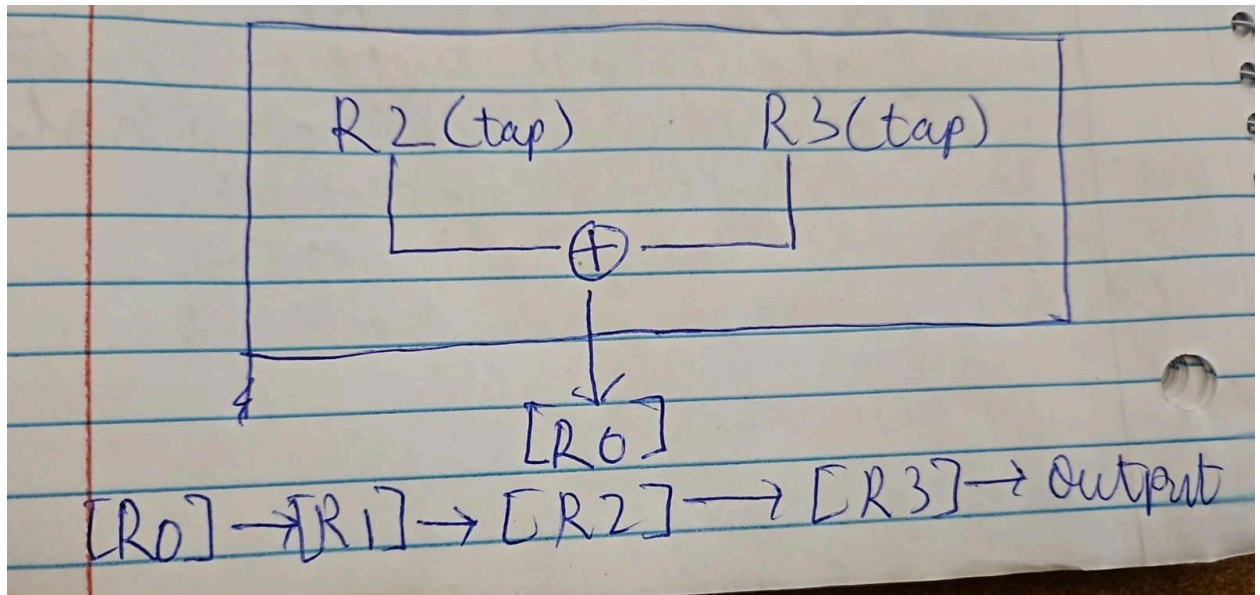
Now, if we look at row 4, and the first bit of row 5, which is 1, we can say that our required registers will register[2] and register[3], since if we XOR register[0] and register[1], we get 0 and not 1.

Row 5:

Now, if we look at row 5, and the first bit of row 6, which is 1, we can say that our required registers will register[2] and register[3], since if we XOR register[0] and register[1], we get 0 and not 1.

Hence, we can say that our final required registers are register[2] and register[3].

6. Draw a diagram of the LFSR configuration you identified in step 5.
(10 points)



7. Translate the plaintext into ASCII.
(15 points)

```
GNU nano 7.2 part02_decrypted_plaintext.txt
Good Morning Julie, the labs where we will be giving bonus points for attendance are Lab03, Lab...>
```

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute	^C Location
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^/ Go To Line

```
GNU nano 7.2 part02_decrypted_plaintext.txt
< Lab... actually we don't know yet... it will be a surprise :)
```

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute	^C Location
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^/ Go To Line

8. Upload your code to canvas as “<netid>_lab04_part01_02.py”. A TA will run your program and make sure it can decrypt the ciphertext. Double check that it runs without any issues before uploading it. Be sure to have both lines in the cycle() to demonstrate the part01 bits and the part02 bits.
(30 points)